

你纠正了 agent 三次——然后呢？

✗ 大多数人：每次被坑就加一条 rule

第 1 次：不要用 any 类型
第 2 次：函数超过 30 行要拆分
第 3 次：不要在组件里直接调 API
.....
第 50 次：agent 完全不读了

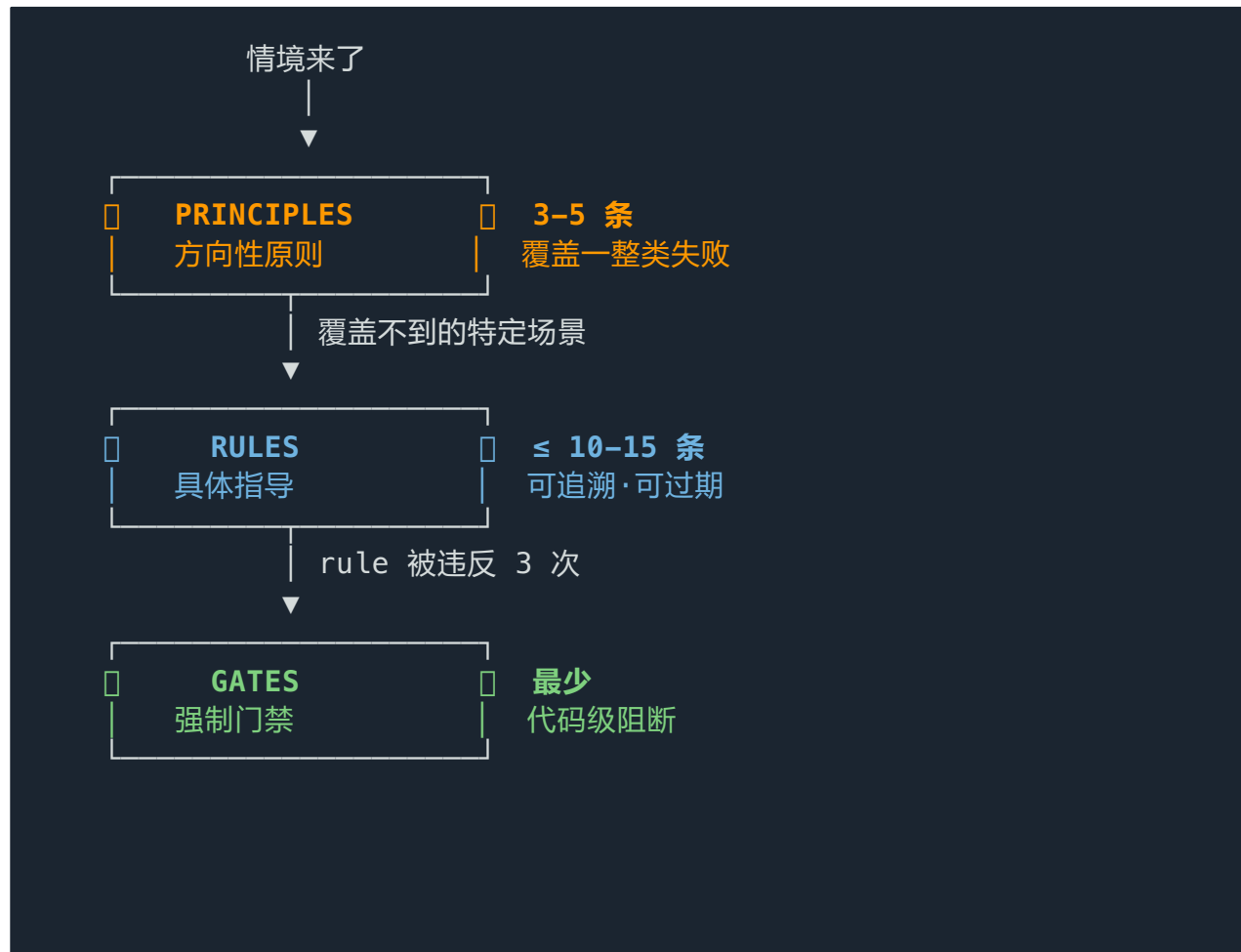
✓ 正确的问题

```
correction #12  丿  
correction #17  □□□→ 同一个偏差根因  
correction #23  丿
```

不是「加什么规则」，而是：
这些纠正背后是同一个什么偏差？

通常，50 条 correction 背后的根因——不超过 3-5 个。

Principles → Rules → Gates: 三层，不是一层



三层共存

不是三选一 —— 三层各有角色，组合起来才覆盖完整

进化向上·吸收

Rule 反复有效 → 吸收进 Principle (Rule 退休)

进化向下·降级

Rule 反复失效 → 降级为 Gate (硬编码阻断)

进化毕业

Gate 30 天不触发 → 上游已经够了 → Gate 毕业

一条好的 Principle 消灭十条 Rule

✖ 坏 (空话)	✔ 好 (可判定)
代码要干净	单函数圈复杂度 ≤ 10 , 违反时必须注释为什么
要认真测试	完成 = 主动尝试破坏它且失败了
注意性能	首屏渲染 $< 200\text{ms}$, 否则视为 P1 bug
安全很重要	所有外部输入必须在入口处验证, 不依赖下游防御

三个标准：抽象到能覆盖一类 · 具体到能判断 · 自带判定标准

① 关于完成度

完成 = 主动破坏且失败, 不是「没发现明显问题」

② 关于数据质量

宁可拒绝, 也不放脏数据进库

③ 关于体验承诺

识别 $< 5\text{s}$ 、首屏 $< 200\text{ms}$ 是可判定的红线

好 principle = 有线 + 有理由 + 有判定标准 —— 三者缺一不可。

每条 Rule 都有出生证明和死亡条件

Rule: 识别接口必须返回 confidence

追溯

- Principle: 数据质量是底线 (P2)
- Evidence: corrections #12, #17, #23

判定标准

- schema 含 confidence: number
- 取值 [0,1]; 不满足 → 400 拒绝

过期条件

- Gate check-schema.sh 部署后退休
- 或连续 30 天无违规重新评估

追溯

对应哪个 principle + 哪些 corrections 催生它

判定标准

怎么判断被遵守了? 必须可判定、无歧义

过期条件

什么时候这条 rule 可以死 (gate 覆盖 / 30 天无违反)

反模式

无追溯 → 孤儿 rule (候选删) · 无过期 → 永远膨胀 · 无证据
→ 想象中的问题

Gates = 代码级阻断：文本约束失败后的硬编码

```
# governance/gates/check-test-coverage.sh
#!/bin/bash
# Gate: 测试覆盖率必须 ≥ 80%
# 追溯: P1 完成=主动破坏且失败
# 证据: Rule R3 被违反 4 次
# 毕业: 连续 60 天不触发

COVERAGE=$(pytest --cov | grep TOTAL ...)
if [ "$COVERAGE" -lt 80 ]; then
    echo "❌ GATE BLOCKED: ${COVERAGE}%<80%"
    exit 1
fi
```

本质

把判定标准变成代码 —— 不是「请遵守」，是「物理上提交不了」

什么时候加

同一 rule 被违反 3 次以上 = 文本约束失效，必须代码化

Gate 毕业

连续 60 天未触发 → 降级为 warning 监控模式 → 退休

警告

到处是 gate = 你的 principles 太弱 / 蒸馏没做好

蒸馏 = 做减法的艺术

操作	触发条件	效果
Rule → Principle 吸收	3+ rules 共享同一根因	Principle 更精炼, Rules 减少
Rule 退休	Gate 已覆盖 / 30 天无违反	Rules 减少
Gate 毕业	60 天未触发	Gates 减少

蒸馏的时机

- corrections.log 积累到 5-10 条时
- 每周定期一次
- 或当感觉「rules 太多了」时

蒸馏不是删除 —— 是升级

蒸馏前: 12 rules

↓ 蒸馏

蒸馏后: 5 rules + 1 new principle

从 10 条 corrections 蒸馏出治理结构

```
CORRECTION 1  跳过 edge case 就说 done
CORRECTION 2  命名 handle_stuff (不具体)
CORRECTION 3  没跑 lint 就提交
CORRECTION 4  改 schema 没更新文档
CORRECTION 5  API key 写在代码里
CORRECTION 6  说「应该没问题」不验证
CORRECTION 7  命名用缩写没人懂
CORRECTION 8  改 3 文件只测 1 个
CORRECTION 9  新增字段没 migration
CORRECTION 10 完成后没自己试一遍
```

完成度不足 (#1,6,8,10)

→ Principle: 完成 = 主动破坏且失败

命名不清晰 (#2,7)

→ Principle: 名字让陌生人秒懂

遗漏配套变更 (#3,4,9)

→ Rule: 主体变更后完成配套 (文档/migration/lint)

安全意识 (#5)

→ Gate: pre-commit 扫描硬编码密钥 (只出现一次也直接上)

10 条 corrections → 2 principles + 1 rule + 1 gate : 文件里 4 条约束, 覆盖 10 种错误变体。

治理结构的文件组织

```
governance/  
├── principles.md           3-5 条, 全放一个文件  
├── rules/  
│   ├── R001-schema.md    每条 rule 一个文件  
│   ├── R002-...  
│   └── _retired/          退休 rules 归档  
│       └── R000-no-any.md 标注退休原因  
└── gates/  
    ├── check-lint.sh      可执行脚本  
    ├── check-secrets.sh  
    └── _graduated/        毕业 gates 归档  
        └── check-xxx.sh
```

Principles 放一个文件

只有 3-5 条, 且需互相参照优先级, 放一起一目了然

Rules 每条一个文件

完整追溯信息 + 独立生命周期, 方便 git 追踪

Gates 是可执行脚本

直接被 pre-commit hook 调用

_retired / _graduated

归档而非删除 —— 保留进化证据

当 Principles 冲突时——优先级解决

Principles (按优先级排序)

P1: 完成 = 主动破坏且失败

P2: 数据质量是底线

(宁可拒绝也不放脏数据)

P3: 每个名字让陌生人秒懂

P4: 用户体验承诺

(识别<5s, 首屏<200ms)

冲突时按序号解决:

P2 > P4 → 数据质量优先于速度

现实: Principles 会冲突

「数据质量是底线」vs「响应<5秒」——验证要10秒呢?

解法: 显式排序

P1 > P2 > P3 > P4, 冲突时按序号决策

这就是品味

选择优先级 = 决定「什么不可妥协」

不同人不同序

→ 不同的 AgentOS → 不同的产出风格, 但都必须有显式排序

Governance 不是独立系统——它是 Knowledge 的产出物

```
Session 互动
  ↓ capture hook
corrections.log (原料)
  ↓ 蒸馏
governance/ (产物)
  principles · rules · gates
  ↓ SessionStart 注入
Agent 下次更好
  ↓
corrections 更少 → 蒸馏更精 → 系统更短
```

Capture 自动 · 执行交给 agent · 审批与排序 —— 是你不可替代的位置。

Governance = 表达层

Knowledge 知道什么、Governance 约束什么

不是配置，是生长

系统从经验中长出约束 —— 不是你想好规则再配进去

你的角色：审批者 + 蒸馏者

capture 自动化 · 蒸馏半自动 (AI 起草你审批) · 排序取舍只有你能做

动手：写你的 Principles + 第一批 Rules

Lab 任务 (30 min)

① governance/principles.md

3-5 条（带优先级排序）；每条：覆盖一类 + 可判定 + 自带判定标准

② governance/rules/

2-3 条（完整格式：追溯 + 判定标准 + 过期条件）

③ governance/gates/

1 个 gate 脚本（lint / test / secrets 检查）

④（可选）用 10 条模拟 corrections 做蒸馏练习

判断点

- 你的 P1 是什么？为什么选它当最高优先级？
- 你的 rule 追溯到哪个 principle？追溯不到 —— 是 rule 有问题还是缺了一条 principle？
- Gate 选什么？为什么这个问题需要代码化阻断而非文本约束？

同一个项目，不同的治理哲学

学员 A

优先级排序

安全 > 质量 > 体验 > 速度

后果预测

agent 花大量时间在安全检查 —— 可能交付慢（医疗/金融对）

学员 B

优先级排序

体验 > 安全 > 质量 > 速度

后果预测

优化 UX 优先 —— 安全边界可能松懈（初创先有人用）

学员 C

优先级排序

完成度第一

后果预测

agent 不易交差 —— 每次交付周期更长（被坑怕了）

这些选择没有对错 —— 但后果可预见。你可以蒸馏调整，但你不能不选：不选 = agent 随机决策。

怎么知道你的治理系统健康不健康

指标	✅ 健康	❌ 生病
Principles 数量	稳定 3-5 条	不断增加
Rules 数量	稳定或减少	持续膨胀
Gates 数量	最少，能毕业	只加不减
Corrections 频率	递减	不变或上升
Rule 退休率	有 rule 在退休	没有 rule 退过
新错命中率	首次就被 principle 覆盖	总需要新 rule

最强信号：新类型错误首次被处理正确 —— principle 覆盖的是「类」，不只是「已知个体」。

Knowledge(2): 三层治理

- ✓ 三层治理: Principles (方向) → Rules (指导) → Gates (强制)
- ✓ Principle: 少、抽象、可判定 | Rule: 可追溯、有证据、可过期 | Gate: 代码化、最后手段、可毕业
- ✓ 蒸馏 = 做减法: Rule → Principle 吸收 · Rule 退休 · Gate 毕业
- ✓ 进化方向: 系统变短、覆盖变广、新错首次对

NEXT ▶

05

Knowledge(3) — Retrieve + Health

governance 怎么注入 agent? 怎么检测健康度?

茶歇 15 分钟 —— 回来讲 Retrieve (注入) + Health (健康检测)。